

SIMATS ENGINEERING

ASSESSMENT TOOL 3

**COURSE NAME: CRYPTOGRAPHY AND NETWORK
SECURITY COURSE CODE: CSA51**

COURSE OUTCOME COVERED

CO2: Analyze Public Key crypto system strategies using RSA and key Exchange for Encryption algorithm to strengthen information security. (BL4,BL5)

Debugging & Optimisation Task : 10%

QUESTIONS: Total Marks: 50 Annexure A

Code of Conduct:

I Sunil Kumar I S (192324155) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

**Annexure A
Debugging & Optimisation Task**

1. Debugging & Optimization of Symmetric Encryption (DES / 3DES Implementation)

A Python program implementing **DES and 3DES encryption** produces inconsistent ciphertext for identical inputs across multiple runs.

- A. Identify and fix logical and implementation errors in key scheduling and permutation steps.

```
from Crypto.Cipher import DES
from Crypto.Util.Padding import pad, unpad
```

```
key = b'12345678'
```

```
# Fixed 8-byte IV (Use random IV in real applications)
iv = b'87654321'
```

```
plaintext = input("Enter Plaintext: ").encode()
```

```
# Encryption
cipher = DES.new(key, DES.MODE_CBC, iv)
```

```

ciphertext = cipher.encrypt(pad(plaintext, DES.block_size))

print("Ciphertext (Hex):", ciphertext.hex())

# Decryption
cipher = DES.new(key, DES.MODE_CBC, iv)
decrypted = unpad(cipher.decrypt(ciphertext), DES.block_size)

print("Decrypted:", decrypted.decode())

```

- B. Optimize the code to reduce execution time for large input datasets (≥ 10 MB).

```

from Crypto.Cipher import DES3
from Crypto.Util.Padding import pad, unpad

# Fixed 24-byte key
key = DES3.adjust_key_parity(b'123456789012345678901234')

# Fixed IV
iv = b'12345678'

plaintext = input("Enter Plaintext: ").encode()

# Encryption
cipher = DES3.new(key, DES3.MODE_CBC, iv)
ciphertext = cipher.encrypt(pad(plaintext, DES3.block_size))

print("Ciphertext (Hex):", ciphertext.hex())

# Decryption
cipher = DES3.new(key, DES3.MODE_CBC, iv)
decrypted = unpad(cipher.decrypt(ciphertext), DES3.block_size)

print("Decrypted:", decrypted.decode())

```

- C. Justify whether replacing DES/3DES with a modern cipher improves both security and performance.

```

from Crypto.Cipher import DES3
from Crypto.Util.Padding import pad

key = DES3.adjust_key_parity(b'123456789012345678901234')
iv = b'12345678'

cipher = DES3.new(key, DES3.MODE_CBC, iv)

BUFFER_SIZE = 1024 * 1024 # 1 MB chunks

with open("input.txt", "rb") as infile, open("encrypted.bin", "wb") as outfile:

    while True:
        chunk = infile.read(BUFFER_SIZE)

        if len(chunk) == 0:
            break

        if len(chunk) % 8 != 0:
            chunk = pad(chunk, 8)

```

```
outfile.write(cipher.encrypt(chunk))
```

```
print("Encryption Completed")
```

2. Performance Analysis of Block Cipher Modes of Operation A Python script implements **AES-like block cipher modes (ECB, CBC, CFB, OFB)** but suffers from high latency and memory inefficiency.

A. Debug incorrect padding and IV handling causing decryption failures. B. Refactor the code to minimize redundant computations and memory usage. C. Compare and optimize the implementation to determine the most efficient mode for real-time secure communication.

```
from Crypto.Cipher import AES
```

```
from Crypto.Util.Padding import pad, unpad
```

```
from Crypto.Random import get_random_bytes
```

```
import time
```

```
# Fixed 16-byte AES key
```

```
key = b'1234567890ABCDEF'
```

```
# Fixed IV (16 bytes) for CBC, CFB, OFB
```

```
iv = b'ABCDEF1234567890'
```

```
plaintext = input("Enter Plaintext: ").encode()
```

```
modes = {
```

```
    "ECB": AES.MODE_ECB,
```

```
    "CBC": AES.MODE_CBC,
```

```
    "CFB": AES.MODE_CFB,
```

```
    "OFB": AES.MODE_OFB
```

```
}
```

```
print("\n==== AES Block Cipher Modes =====")
```

```
for name, mode in modes.items():
```

```

print("\n-----")
print("Mode:", name)

start = time.perf_counter()

# Encryption
if mode == AES.MODE_ECB:
    cipher = AES.new(key, mode)
    ciphertext = cipher.encrypt(pad(plaintext, AES.block_size))
elif mode == AES.MODE_CBC:
    cipher = AES.new(key, mode, iv)
    ciphertext = cipher.encrypt(pad(plaintext, AES.block_size))
else:
    cipher = AES.new(key, mode, iv)
    ciphertext = cipher.encrypt(plaintext)

enc_time = time.perf_counter() - start

# Decryption
if mode == AES.MODE_ECB:
    cipher = AES.new(key, mode)
    decrypted = unpad(cipher.decrypt(ciphertext), AES.block_size)
elif mode == AES.MODE_CBC:
    cipher = AES.new(key, mode, iv)
    decrypted = unpad(cipher.decrypt(ciphertext), AES.block_size)
else:
    cipher = AES.new(key, mode, iv)
    decrypted = cipher.decrypt(ciphertext)

print("Ciphertext :", ciphertext.hex())

```

```

print("Decrypted :", decrypted.decode())

print("Time Taken :", round(enc_time * 1000, 6), "ms")


print("\n=====")
print("Performance Summary")
print("ECB : Fast but insecure (not recommended)")
print("CBC : Secure, moderate speed")
print("CFB : Fast, no padding, suitable for real-time communication")
print("OFB : Fast, no error propagation, suitable for streaming")
print("\nBest Mode: CFB/OFB for real-time secure communication.")

```

3. Debugging RSA Cryptosystem Implementation A Python-based **RSA encryption system** fails when encrypting large messages and occasionally produces incorrect plaintext after decryption.

A. Identify issues in prime generation, key generation, and modular exponentiation.

```

from Crypto.PublicKey import RSA
from Crypto.Cipher import PKCS1_OAEP

# Generate 2048-bit RSA Key Pair
key = RSA.generate(2048)

public_key = key.publickey()
private_key = key

encryptor = PKCS1_OAEP.new(public_key)
decryptor = PKCS1_OAEP.new(private_key)

message = input("Enter Message: ").encode()

# Encryption
ciphertext = encryptor.encrypt(message)

```

```
print("Ciphertext:")
print(ciphertext.hex())
```

```
# Decryption
```

```
plaintext = decryptor.decrypt(ciphertext)
```

```
print("Recovered Plaintext:")
print(plaintext.decode())
```

B. Optimize the algorithm using efficient techniques (e.g., fast exponentiation, Chinese Remainder Theorem).

```
from Crypto.PublicKey import RSA
from Crypto.Cipher import PKCS1_OAEP
import time
```

```
key = RSA.generate(2048)
```

```
public_key = key.publickey()
private_key = key
```

```
encryptor = PKCS1_OAEP.new(public_key)
decryptor = PKCS1_OAEP.new(private_key)
```

```
message = input("Enter Message: ").encode()
```

```
# Encryption Timing
start = time.perf_counter()
ciphertext = encryptor.encrypt(message)
enc_time = time.perf_counter() - start
```

```
# Decryption Timing
start = time.perf_counter()
plaintext = decryptor.decrypt(ciphertext)
dec_time = time.perf_counter() - start
```

```
print("Recovered Message:", plaintext.decode())
print("Encryption Time:", round(enc_time * 1000, 6), "ms")
print("Decryption Time:", round(dec_time * 1000, 6), "ms")
```

```
print("\nOptimizations Used:")
print("✓ Fast Modular Exponentiation")
print("✓ Chinese Remainder Theorem (CRT)")
print("✓ OAEP Padding")
```

D. Evaluate the trade-offs between key size, security, and computational performance.

```
key_sizes = [1024, 2048, 3072, 4096]
```

```
print("RSA Key Size Comparison\n")
```

```

for size in key_sizes:

    print("Key Size:", size)

    if size == 1024:
        print("Security   : Low")
        print("Performance : Very Fast")

    elif size == 2048:
        print("Security   : High (Recommended)")
        print("Performance : Fast")

    elif size == 3072:
        print("Security   : Very High")
        print("Performance : Moderate")

    elif size == 4096:
        print("Security   : Extremely High")
        print("Performance : Slow")

    print("-----")

    print("Best Choice: 2048-bit RSA")
    print("Reason: Provides the best balance between security and
    computational performance.")

```

4. Optimization and Security Analysis of Diffie-Hellman Key Exchange A Python implementation of the **Diffie-Hellman key exchange** is vulnerable to performance bottlenecks and potential security flaws.

A. Debug incorrect shared key generation due to improper modular arithmetic.

Debugging Diffie-Hellman Key Exchange

Public Values

p = 23

g = 5

Private Keys

a = 6

b = 15

Public Keys

A = pow(g, a, p)

B = pow(g, b, p)

```
print("Alice Public Key :", A)
print("Bob Public Key  :", B)
```

```
# Shared Secret
```

```
Alice_Key = pow(B, a, p)
```

```
Bob_Key = pow(A, b, p)
```

```
print("Alice Shared Key :", Alice_Key)
```

```
print("Bob Shared Key  :", Bob_Key)
```

```
if Alice_Key == Bob_Key:
```

```
    print("Shared Key Generated Successfully")
```

```
else:
```

```
    print("Shared Key Generation Failed")
```

B. Optimize exponentiation operations for large prime values.

```
import time
```

```
# Large Prime
```

```
p = 104729
```

```
g = 2
```

```
a = 12345
```

```
b = 54321
```

```
start = time.perf_counter()
```

```
A = pow(g, a, p)
```

```
B = pow(g, b, p)
```

```
Alice_Key = pow(B, a, p)
```

```
Bob_Key = pow(A, b, p)
```

```
end = time.perf_counter()
```

```
print("Shared Key:", Alice_Key)
```

```
print("Execution Time:", round((end-start)*1000,6),"ms")
```

```
print("\nOptimization Used")
```

```
print("✓ Fast Modular Exponentiation using pow(base, exponent, modulus)")
```

C. Extend the program to mitigate man-in-the-middle attacks and evaluate the performance impact of your enhancements.

```
import hashlib
```

```
p = 23
```

```
g = 5
```

```
a = 6
```

```
b = 15
```

```
A = pow(g, a, p)
```

```
B = pow(g, b, p)
```

```
Alice_Key = pow(B, a, p)
```

```
Bob_Key = pow(A, b, p)
```

```
# Authentication using Hash
```

```
alice_hash = hashlib.sha256(str(Alice_Key).encode()).hexdigest()
```

```
bob_hash = hashlib.sha256(str(Bob_Key).encode()).hexdigest()
```

```
print("Alice Hash :", alice_hash)
```

```
print("Bob Hash  :", bob_hash)
```

```
if alice_hash == bob_hash:
```

```
    print("Authentication Successful")
```

```
    print("No Man-in-the-Middle Attack")
```

```
else:
```

```
    print("Possible MITM Attack Detected")
```

```
print("\nPerformance Impact")
```

```
print("Hash verification adds very small overhead but greatly improves  
security.")
```

5. Debugging and Enhancing Elliptic Curve Cryptography (ECC)

Implementation A Python script implementing **Elliptic Curve Cryptography** for key exchange produces incorrect results for certain curve parameters.

A. Identify and fix errors in point addition and scalar multiplication logic.

```
from tinyec import registry
```

```
# Select Curve
```

```
curve = registry.get_curve('brainpoolP256r1')
```

```
# Private Keys
```

```
alice_private = 1234
```

```
bob_private = 5678
```

```
# Public Keys
```

```
alice_public = alice_private * curve.g
```

```
bob_public = bob_private * curve.g
```

```
# Shared Secret
```

```
alice_shared = alice_private * bob_public
```

```
bob_shared = bob_private * alice_public
```

```
print("Alice Public Key:", alice_public)
```

```
print("Bob Public Key:", bob_public)
```

```
print("\nAlice Shared Secret:")
```

```
print(alice_shared)
```

```
print("\nBob Shared Secret:")
```

```
print(bob_shared)
```

```
if alice_shared == bob_shared:
```

```
    print("\nShared Secret Generated Successfully")
```

```
else:
```

```
print("\nError in ECC Implementation")
```

B. Optimize the implementation using efficient algorithms (e.g., double-and-add).

```
from tinyec import registry
```

```
import time
```

```
curve = registry.get_curve('brainpoolP256r1')
```

```
private_key = 12345
```

```
start = time.perf_counter()
```

```
# tinyec internally uses efficient scalar multiplication
```

```
public_key = private_key * curve.g
```

```
end = time.perf_counter()
```

```
print("Public Key:")
```

```
print(public_key)
```

```
print("\nExecution Time:", round((end-start)*1000,6), "ms")
```

```
print("\nOptimization Used:")
```

```
print("✓ Double-and-Add Scalar Multiplication")
```

C. Compare the optimized ECC implementation with RSA in terms of computational efficiency and security strength, and justify suitability for resource-constrained environments.

```
from tinyec import registry
```

```
import time
```

```
curve = registry.get_curve('brainpoolP256r1')
```

```
private_key = 12345
```

```
start = time.perf_counter()
```

```

# tinyec internally uses efficient scalar multiplication
public_key = private_key * curve.g

end = time.perf_counter()

print("Public Key:")
print(public_key)

print("\nExecution Time:", round((end-start)*1000,6), "ms")

print("\nOptimization Used:")
print("✓ Double-and-Add Scalar Multiplication")

```

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Identification	20%	Accurately identifies all errors and root causes with clear understanding	Identifies most errors with minor gaps	Identifies some errors but misses key issues	Unable to identify errors correctly
Debugging	25%	Applies systematic debugging techniques effectively to resolve issues	Uses appropriate debugging methods with minor errors	Limited debugging approach; requires guidance	Unable to debug or resolve issues
Optimization	20%	Optimizes code by significantly improving time complexity using efficient algorithms	Improves time complexity with minor gaps in efficiency	Limited improvement in time complexity	No improvement; inefficient approach retained

Analysis	20%	Demonstrate strong logical reasoning and clear justification of solutions	Shows reasonable analysis with some justification	Limited analysis; weak reasoning	No analytical reasoning demonstrated
Code Quality	15%	Produces clean, well structured code with proper comments and documentation	Code is mostly clear with minor documentation gaps	Code lacks structure and proper documentation	Poorly written code with no documentation